



华中农业大学

HUAZHONG AGRICULTURAL UNIVERSITY

综合实训报告

转转校园二手物品交易平台设计与实现

姓名：张可凡、阮崇轩、刘轩霆、胡欣悦、李硕

学号：2024317220511 、
2023301210209 、
2024317220417 、
2024317220423 、
2024317220419

班级：计科 2403、2401 班

指导老师：徐士伟

中国 武汉

二〇二六年七月

2026.07

目录

1	实训目的及内容	2
1.1	实训目的	2
1.2	实训内容	2
1.3	实训安排	2
2	需求分析	3
2.1	功能需求	4
2.2	非功能需求	5
3	系统设计	6
3.1	体系结构设计	6
3.2	功能设计	7
3.3	数据设计	12
4	系统实现	14
4.1	JWT 双 Token 认证机制	14
4.2	订单状态流转引擎	15
4.3	商品搜索与筛选	15
4.4	消息即时通信	16
4.5	数据统计与可视化	16
5	系统测试	16
5.1	测试策略	16
5.2	测试用例设计	16
5.3	测试结果	16
6	结论与体会	19
6.1	结论	19
6.2	个人体会	20
7	参考文献	20

1 实训目的及内容

1.1 实训目的

本次综合实训以软件工程理论为指导，通过开发一个完整的校园二手物品交易平台，达到以下目的：

1. 巩固和深化软件工程、Web 前端开发技术、Java 程序设计、数据库原理与应用、计算机网络等课程的理论知识。
2. 掌握前后端分离架构的开发模式，熟悉 SpringBoot + Vue3 主流技术栈的工程化实践。
3. 培养团队协作能力与项目管理能力，体验从需求分析、系统设计、编码实现到测试部署的完整软件生命周期。
4. 提升独立解决实际问题的能力，包括技术选型、架构设计、性能优化、安全防护等综合工程素养。

1.2 实训内容

本项目”转转”是一个面向高校学生的 C2C 二手物品交易 Web 应用系统，旨在解决校园闲置物品循环利用的需求。系统采用前后端分离架构，后端基于 Spring Boot 3.2.0 + Spring Security + JWT + MyBatis-Plus 3.5.5 实现 RESTful API，JDK 17 运行环境；前端基于 Vue 3.4 + TypeScript 5.3 + Vite 5 + Element Plus 2.5 构建用户界面；使用 MySQL 8.0 存储业务数据，Redis 7 提供缓存加速，全容器化 Docker Compose 一键部署。

系统实现了完整的交易闭环，包括用户管理、商品管理、购物车与订单交易、站内消息沟通、后台管理五大核心模块，共计 60 余个 RESTful API 和 20 余个前端页面，并提供 Docker Compose 一键部署方案。

1.3 实训安排

1.3.1 团队组成

实训小组共 5 人，分工如表 1 所示：

1.3.2 开发计划

项目周期为 2026 年 7 月 4 日至 2026 年 7 月 10 日，共分四个阶段：

- **第 1 天（7 月 4 日）：**环境搭建与基础框架——Git 仓库创建、Docker 编排、前后端项目脚手架、数据库表设计。
- **第 2-3 天（7 月 5 日-6 日）：**核心功能开发（用户 + 商品）——安全框架集成、注册登录、商品 CRUD、分类与收藏。
- **第 4-5 天（7 月 7 日-8 日）：**核心功能开发（订单 + 消息）——购物车、订单状态流转、站内信、系统通知。

表 1: 团队成员与角色分工

成员	角色	职责
张可凡（队长）	全栈/项目管理	Docker 部署、管理后台前端、Postman 测试、文档整合
阮崇轩	前端开发	前端工程搭建、用户认证模块、消息通知系统、UI 动效与交互优化
刘轩霆	后端开发 A	用户模块、商品模块、安全框架、订单核心流程、后台管理 API
胡欣悦	后端开发 B	购物车模块、消息与通知模块、数据统计、文件上传、Docker 部署运维
李硕	前端开发	首页与商品模块、购物车与订单页面、管理后台页面、数据可视化

- 第 6-7 天（7 月 9 日-10 日）：管理后台 + 联调测试——用户/商品/订单管理、数据统计、全量接口联调、文档编写。

2 需求分析

转转校园二手交易平台作为面向高校学生的 C2C 交易平台，主要服务三类用户角色：游客（未登录用户）、普通用户（买家和卖家双重身份）以及管理员。系统的顶层用例如图1所示。

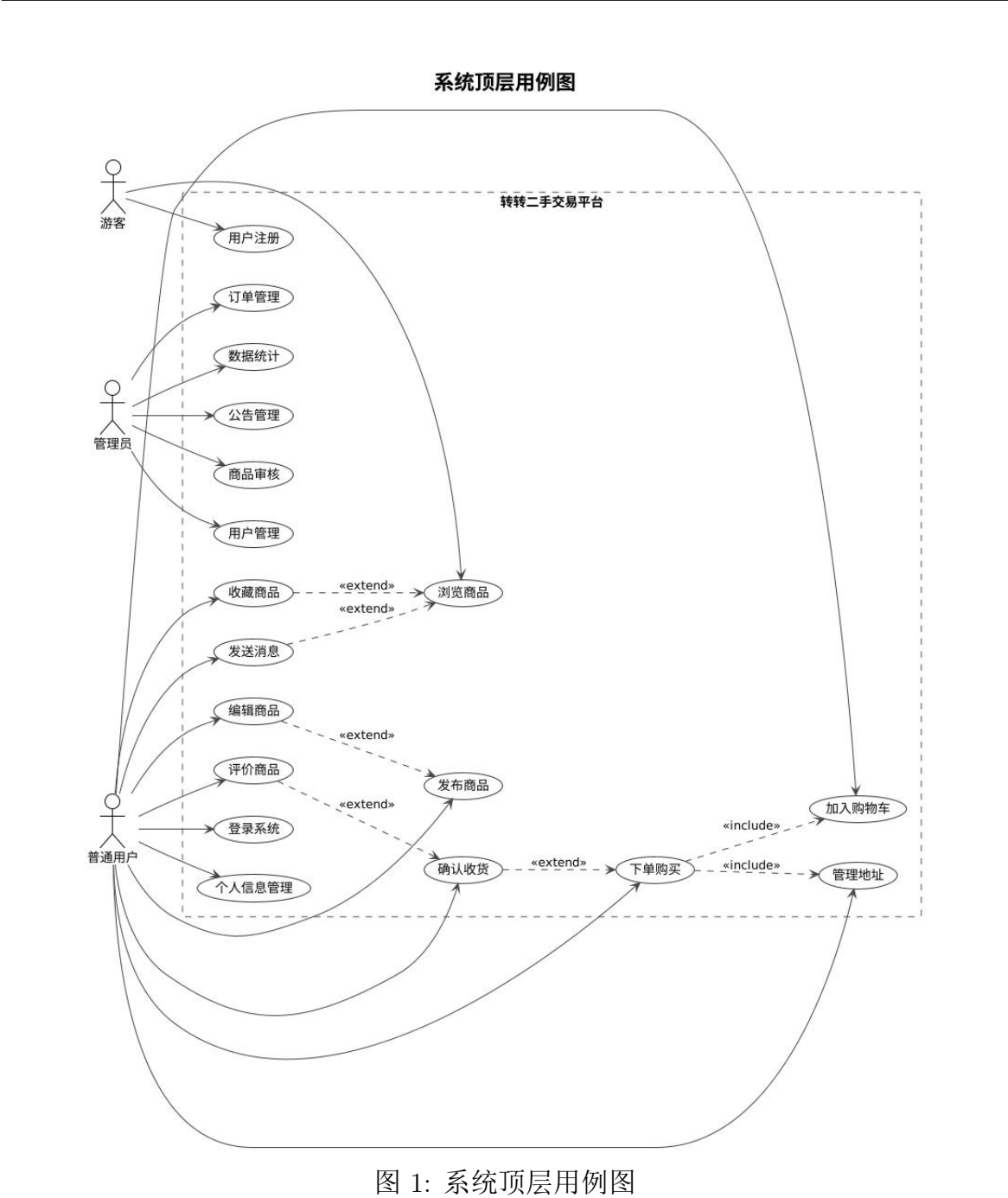


图 1: 系统顶层用例图

2.1 功能需求

系统包含以下五大核心功能模块：

2.1.1 用户管理模块

支持用户的完整生命周期管理。游客可通过邮箱/手机号验证码注册成为平台用户，注册后可使用用户名或邮箱或手机号登录系统。系统采用 JWT 双 Token 机制（Access Token 2 小时过期，Refresh Token 7 天过期）实现无状态认证，支持密码找回、Token 无感刷新、安全退出（Token 加入 Redis 黑名单）等功能。用户可在个人中心编辑昵称、简介、性别，上传头像，管理多个收货地址并设置默认地址。

2.1.2 商品管理模块

支持商品的完整生命周期管理。卖家可发布商品，填写标题、描述（富文本）、价格、原价、成色，选择分类并上传多张图片。商品发布后需管理员审核方可上架展示。买家可通过首页推荐、分类导航、关键词搜索（全文索引）浏览商品，支持按分类、价格区间、成色筛选，按价格、时间、热度排序。买家可收藏感兴趣的物品，卖家可查看和管理自己发布的商品。

2.1.3 订单交易模块

实现完整的交易闭环。买家可将商品加入购物车统一结算，下单时选择收货地址。订单状态按“待付款 → 待发货 → 待收货 → 已完成”流转，系统提供模拟支付功能，卖家可录入物流信息发货，买家确认收货后可对交易进行评分和评价。所有状态变更均记录订单日志，便于追踪溯源。

2.1.4 消息与通知模块

支持买卖双方的即时沟通。买家可通过商品详情页的“联系卖家”发起会话，双方可互发文本消息或图片消息，支持会话列表查看、聊天记录分页查询、消息已读/未读标记和未读计数。系统自动推送订单状态变更、审核结果等系统通知。

2.1.5 后台管理模块

为管理员提供全面的管理功能。包括用户管理（用户列表、搜索筛选、禁用/启用、修改角色）、商品审核（待审核列表、审核通过/驳回、违规强制下架）、订单管理（按状态/单号/日期查询、异常状态处理）、数据统计（注册用户量、商品发布量、成交量、分类占比饼图、趋势折线图，基于 ECharts 实现）以及公告管理（发布/编辑/删除系统公告）。

2.2 非功能需求

- **安全性：**密码 BCrypt 加密存储，JWT 双 Token 无状态认证，接口 RBAC 权限控制，SQL 注入防护（排序字段白名单校验），Nginx 安全响应头配置（X-Frame-Options/XSS-Protection/Content-Type-Options），API 限流防护（全局 30r/s、登录接口 10r/m）。
- **性能：**Redis 缓存验证码、黑名单 Token 及热点商品数据，数据库 4 个复合索引优化高频查询，N+1 查询问题修复，前端 Gzip 压缩与路由懒加载。
- **可用性：**响应式 UI 设计兼容移动端与桌面端，统一异常处理与友好的错误提示，Token 自动刷新机制保证会话连续性，Docker Compose 一键部署与恢复。
- **可维护性：**前后端分离架构，RESTful API 规范，Knife4j 自动生成接口文档，统一返回结果封装，完善的订单日志审计。

3 系统设计

3.1 体系结构设计

系统采用前后端分离的 B/S 架构，前端与后端通过 RESTful API 进行数据交互。整体架构分为四层，如图2所示。

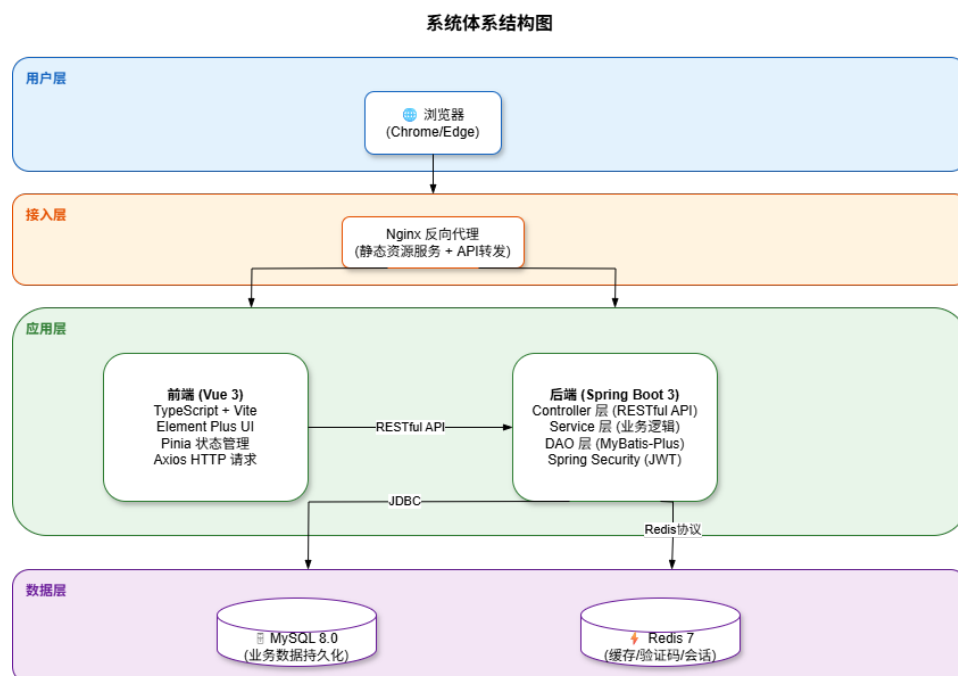


图 2: 系统体系结构图

3.1.1 前端架构

前端基于 Vue 3.4 + TypeScript 5.3 + Vite 5 构建，采用组合式 API (Composition API) 开发模式。使用 Element Plus 2.5 提供 UI 组件库，Pinia 2.1 (结合持久化插件 pinia-plugin-persistedstate) 管理全局状态，Vue Router 4.2 实现路由控制，共配置 21 条路由，全部采用懒加载方式。Axios 封装请求拦截器自动携带 JWT Token，响应拦截器统一处理错误并支持 Token 无感自动刷新。前端构建通过 unplugin-auto-import 和 unplugin-vue-components 实现 Element Plus 按需自动导入，使用 vite-plugin-compression 开启 Gzip 压缩。

3.1.2 后端架构

后端采用 Spring Boot 3.2.0 + MyBatis-Plus 3.5.5 + Spring Security 技术栈，JDK 17 运行环境，遵循经典的三层架构设计：

- **Controller 层**：接收 HTTP 请求，参数校验，调用 Service 层处理业务，返回统一封装的 Result 对象。API 文档通过 Knife4j (Swagger 增强) 自动生成。
- **Service 层**：业务逻辑处理，包括用户认证、商品管理、订单流转、消息通信等核心业务。
- **DAO 层**：基于 MyBatis-Plus 操作数据库，支持 LambdaQueryWrapper 动态条件查询和 MyBatis-Plus 分页插件。

安全方面, Spring Security 配置为无状态会话模式, 通过 JwtAuthenticationFilter 拦截请求, 从请求头提取 Token, 验证有效性并检查 Redis 黑名单后设置 Security-Context。项目使用 Hutool 工具库 (5.8.25) 提供通用工具方法, Lombok 简化实体代码。

3.1.3 部署架构

系统采用 Docker Compose 编排五容器架构: Nginx (前端静态资源服务与反向代理, 映射端口 3000)、Spring Boot 后端、MySQL 8.0 数据库、Redis 7 缓存以及 phpMyAdmin 数据库管理工具 (映射端口 3001)。所有服务通过内部桥接网络通信, MySQL 和 Redis 不对外暴露端口, 后端仅通过 Nginx 反向代理对外暴露。使用数据卷持久化 MySQL 数据和 Redis 数据, 确保容器重启不丢失。

3.2 功能设计

系统五大核心功能模块之间相互协作, 共同构成完整的交易闭环。模块之间的调用关系如图3所示。

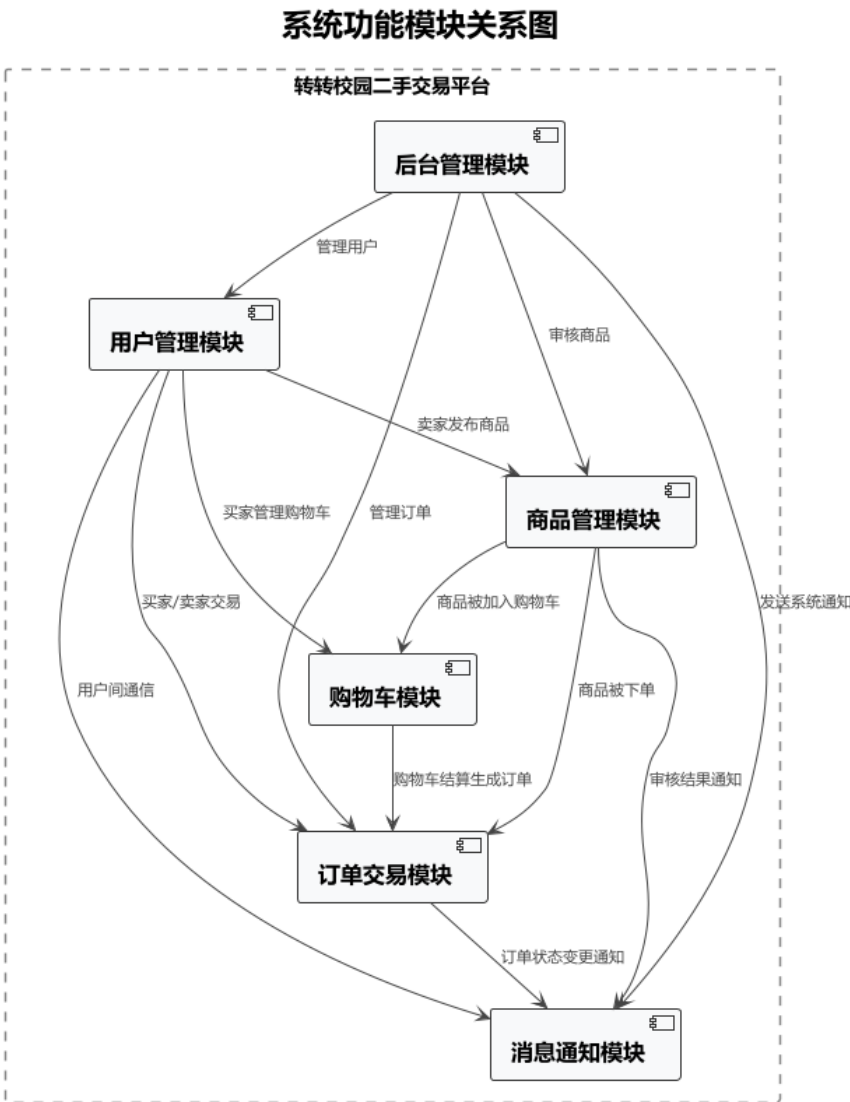


图 3: 功能模块关系图

3.2.1 用户认证模块流程

用户认证模块涵盖注册、登录、Token 刷新、密码找回、退出登录等核心流程。用户注册后获取 JWT 双 Token，后续请求携带 Access Token 访问受保护资源，Token 过期后由 Refresh Token 无感刷新。图4展示了用户认证的完整流程。

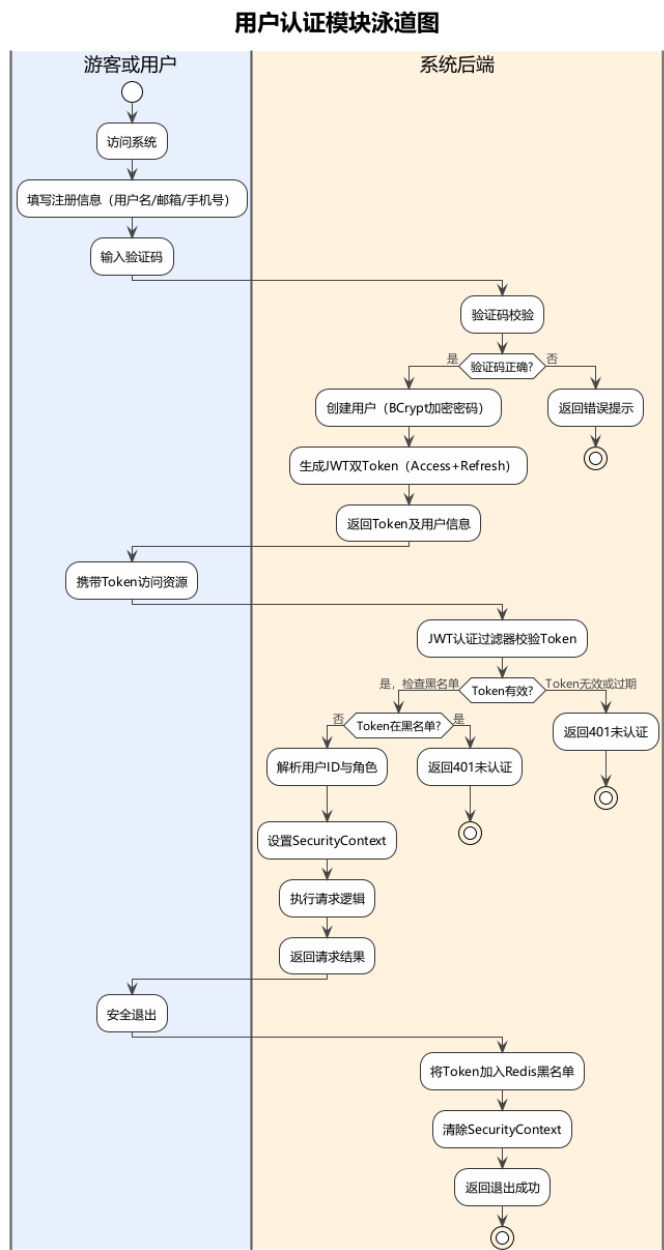


图 4: 用户认证模块泳道图

3.2.2 商品管理模块流程

商品管理模块支持商品的完整生命周期——卖家发布商品后需管理员审核方可上架，买家通过搜索、分类浏览、收藏等方式发现商品。图5展示了商品从发布到上架的跨角色协作流程。

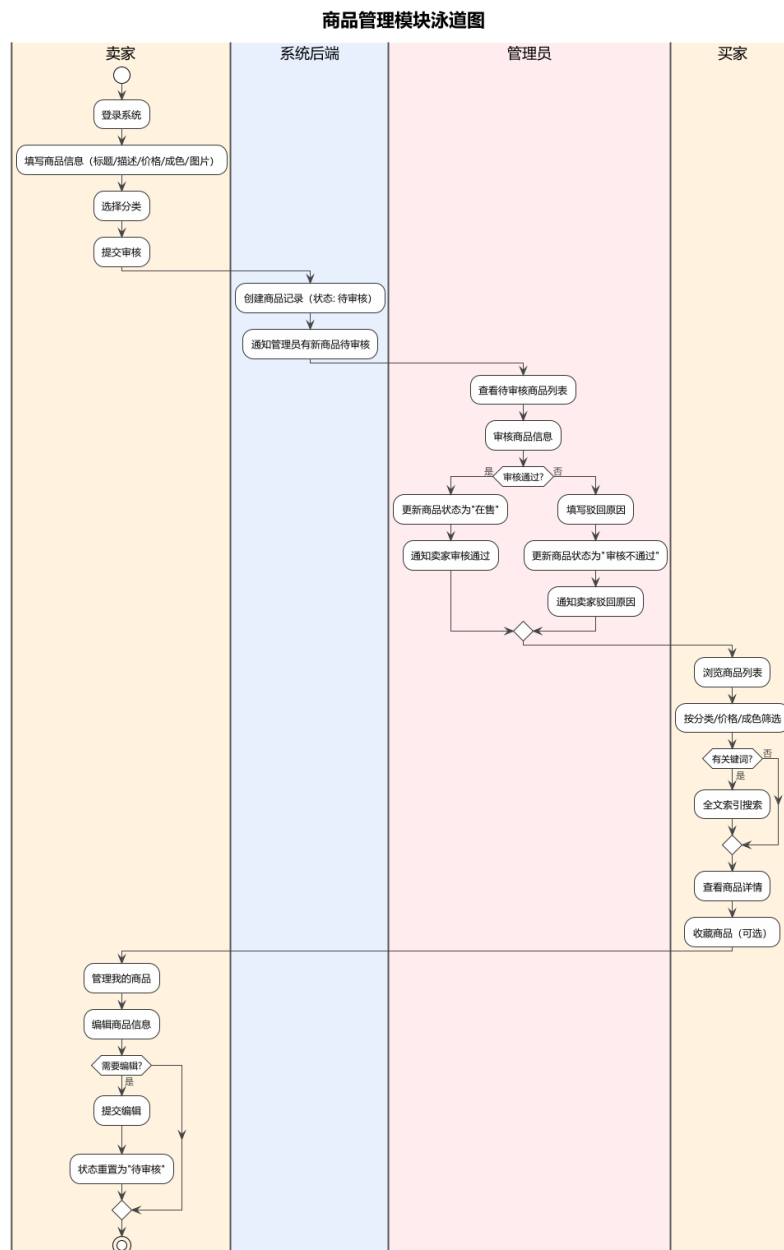


图 5: 商品管理模块泳道图

3.2.3 订单交易模块流程

订单交易模块实现了从购物车、下单、支付、发货、收货到评价的完整交易闭环。买卖双方围绕订单状态流转进行协作，所有状态变更均记录订单日志。图6展示了订单交易的跨角色流程。

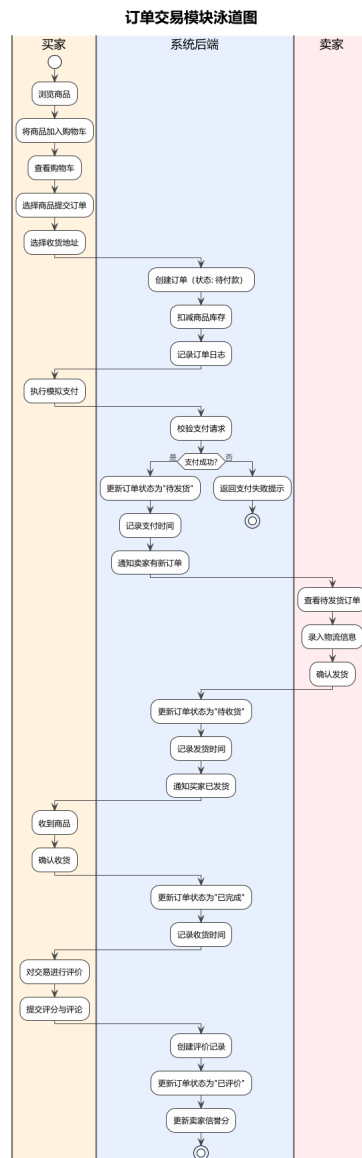


图 6: 订单交易模块泳道图

3.2.4 消息通知模块流程

消息通知模块支持买卖双方之间的站内即时通信，以及系统向用户推送订单状态变更、审核结果等通知。图7展示了消息发送与通知推送的流程。

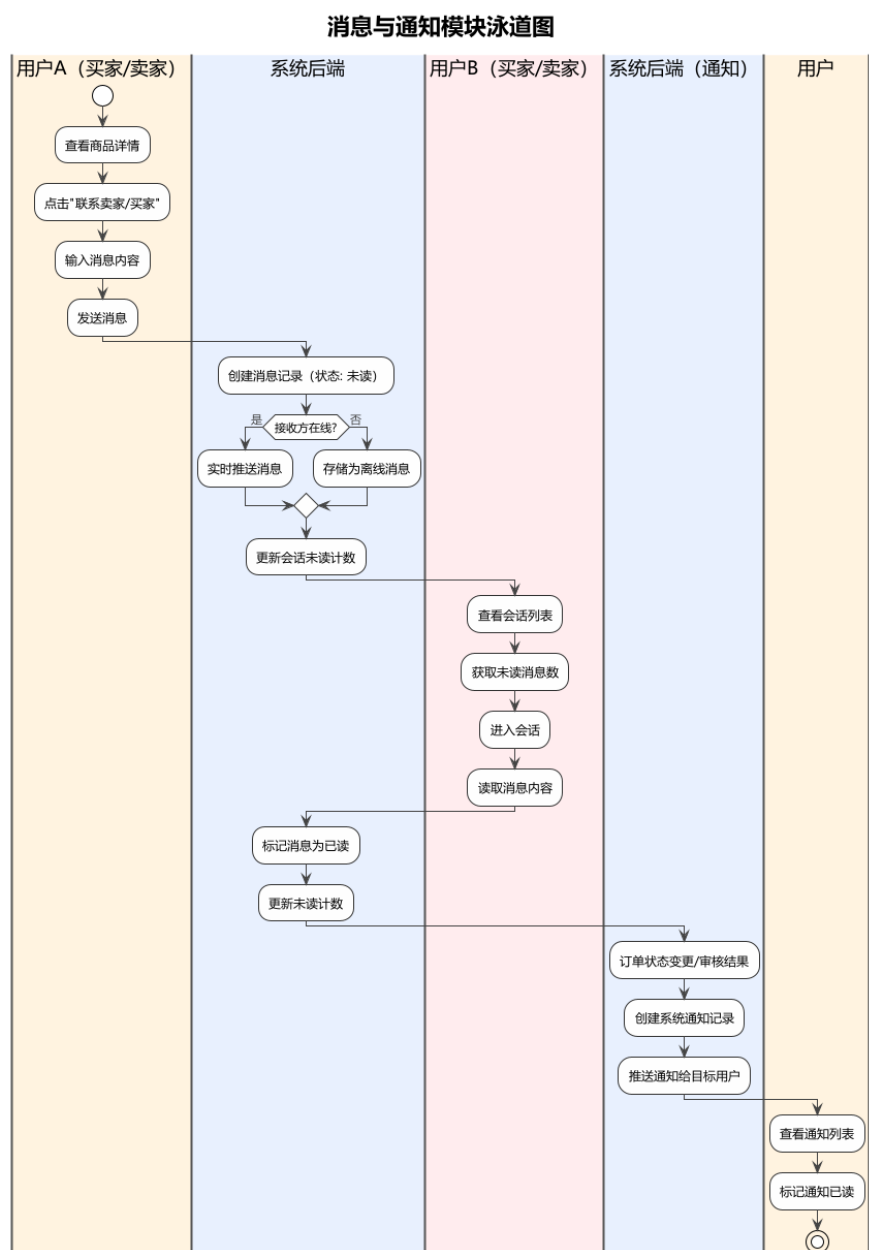


图 7: 消息通知模块泳道图

3.2.5 后台管理模块流程

后台管理模块为管理员提供用户管理、商品审核、订单管理、数据统计和公告管理等全面管理功能。图8展示了管理员执行审核与管理的流程。

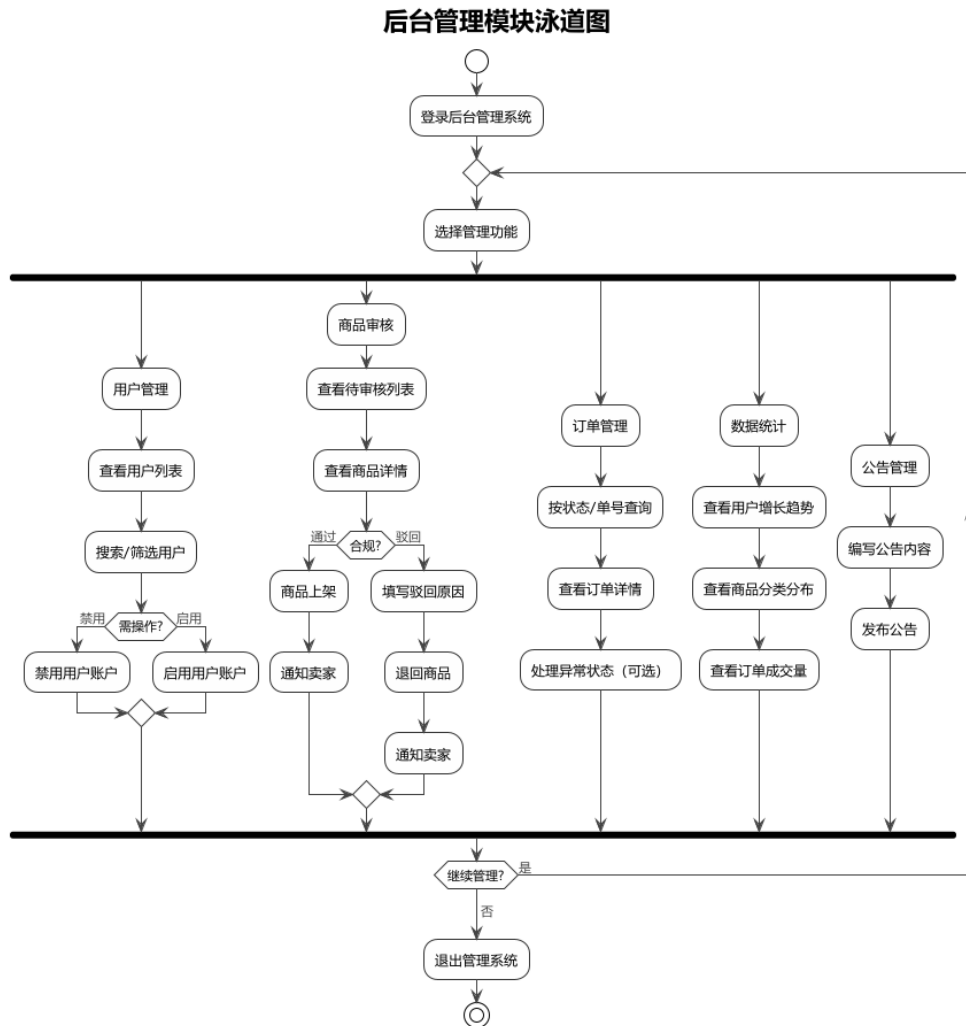


图 8: 后台管理模块泳道图

3.3 数据设计

系统共设计 13 张核心业务表，涵盖用户、商品、订单、消息等全部业务实体。表结构和关系如图9所示。

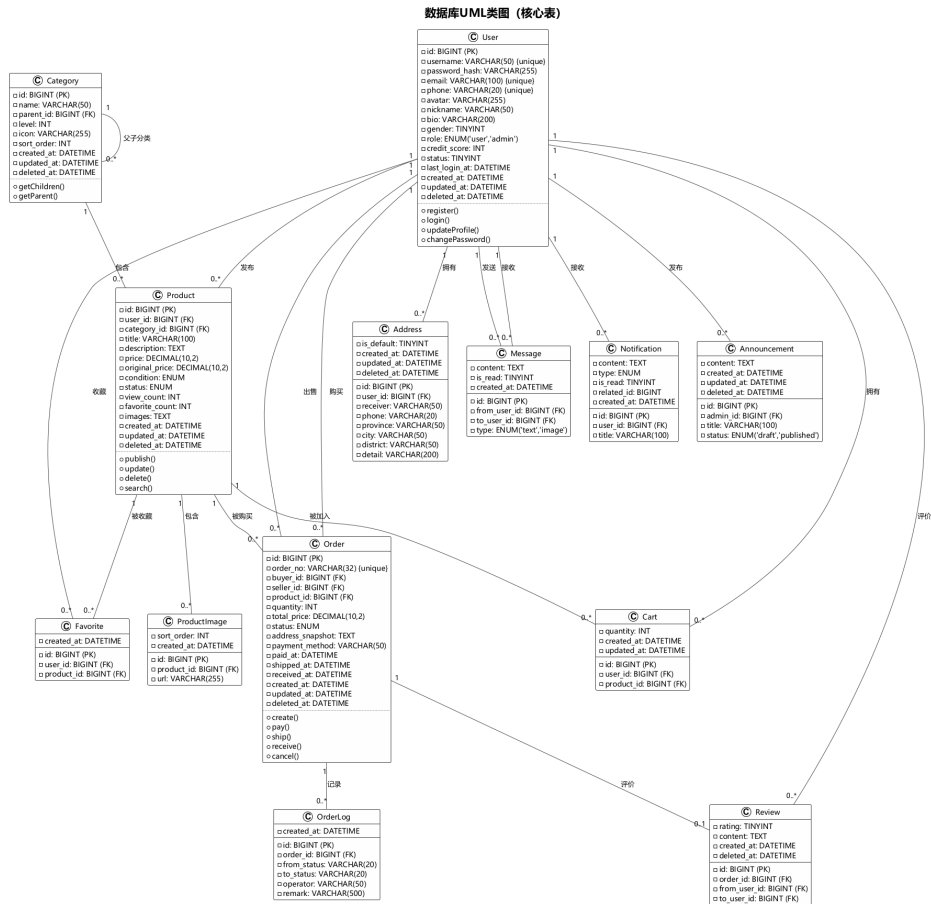


图 9: 数据库 UML 类图（核心表）

3.3.1 核心数据库表

数据表设计特点：所有表使用 InnoDB 引擎和 utf8mb4 字符集；采用软删除策略（deleted_at 字段配合 MyBatis-Plus @TableLogic）；created_at 和 updated_at 通过 MyMetaObjectHandler 自动填充；product 表含全文索引支持商品搜索；通过复合索引优化消息会话查询、未读计数、收藏排序等高频查询场景。

表 2: 核心数据库表清单

表名	用途	关键字段
user	用户表	username, password_hash, email, phone, role
category	商品分类表	name, parent_id, level
product	商品表	user_id, category_id, title, price, status
product_image	商品图片表	product_id, url, sort_order
address	收货地址表	user_id, receiver, phone, province, city
order	订单表	order_no, buyer_id, seller_id, status
order_log	订单日志表	order_id, from_status, to_status
cart	购物车表	user_id, product_id, quantity
message	消息表	from_user_id, to_user_id, content, type
favorite	收藏表	user_id, product_id
review	评价表	order_id, from_user_id, rating
notification	通知表	user_id, title, content, type
announcement	公告表	admin_id, title, content, status

4 系统实现

4.1 JWT 双 Token 认证机制

系统采用 Access Token + Refresh Token 双 Token 机制实现无状态认证。Access Token 有效期为 2 小时，存储用户 ID 和角色信息；Refresh Token 有效期为 7 天，用于无感刷新 Token。

Listing 1: JWT 认证过滤器核心逻辑

```
@Override
protected void doFilterInternal(HttpServletRequest request,
    HttpServletResponse response, FilterChain filterChain)
    throws ServletException, IOException {
    String token = extractToken(request);
    if (StringUtils.hasText(token) && jwtUtil.validateToken(token)
        && !jwtUtil.isRefreshToken(token)) {
        Long userId = jwtUtil.getUserIdFromToken(token);
        // check blacklist
        String tokenId = jwtUtil.getTokenId(token);
        if (tokenId != null && Boolean.TRUE.equals(
            redisTemplate.hasKey("blacklist:token:" + tokenId))) {
            filterChain.doFilter(request, response);
            return;
        }
        // reload user from database for role verification
        User user = userMapper.selectById(userId);
```

```

        if (user == null || user.getStatus() == 0) {
            filterChain.doFilter(request, response);
            return;
        }
        // set authentication with DB role
        UsernamePasswordAuthenticationToken authentication =
            new UsernamePasswordAuthenticationToken(
                userId, null,
                Collections.singletonList(
                    () -> "ROLE_" + user.getRole().toUpperCase()));
        SecurityContextHolder.getContext()
            .setAuthentication(authentication);
    }
    filterChain.doFilter(request, response);
}

private String extractToken(HttpServletRequest request) {
    String bearerToken = request.getHeader("Authorization");
    if (StringUtils.hasText(bearerToken)
        && bearerToken.startsWith("Bearer_")) {
        return bearerToken.substring(7);
    }
    return null;
}
}

```

4.2 订单状态流转引擎

订单模块是整个系统的核心业务，涉及多个状态的流转和数据一致性。采用状态模式设计，每个状态变更均记录订单日志（order_log 表），关键操作（创建订单扣减库存）通过 @Transactional 保证原子性。此外，系统采用模拟支付流程，无需接入真实第三方支付即可完整体验交易闭环。

订单状态流转路径为：

待付款 $\xrightarrow{\text{支付}}$ 待发货 $\xrightarrow{\text{发货}}$ 待收货 $\xrightarrow{\text{收货}}$ 已完成 $\xrightarrow{\text{评价}}$ 已评价

4.3 商品搜索与筛选

商品模块支持多条件组合查询，包括分类筛选、价格区间、成色筛选、关键词搜索以及按价格/时间/热度排序。基于 MyBatis-Plus 的 LambdaQueryWrapper 动态拼接查询条件，配合 MySQL 全文索引实现关键词搜索。为防范 SQL 注入，排序字段采用白名单机制，仅允许预定义字段参与排序。

订单状态流转图

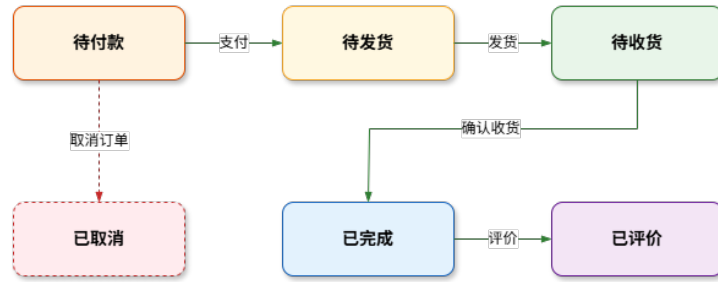


图 10: 订单状态流转图

4.4 消息即时通信

站内信模块实现了买家与卖家的实时沟通功能。消息表记录所有通信内容，按会话（Conversation）维度组织，通过 `from_user_id` 和 `to_user_id` 的组合查询生成会话列表。系统支持消息已读/未读标记，并通过 Redis 缓存未读计数提升查询性能。

4.5 数据统计与可视化

管理后台的数据统计模块基于 ECharts 实现数据可视化，包括用户增长趋势折线图、商品分类分布饼图、每日订单成交量柱状图等。后端通过 SQL 聚合查询获取统计数据，前端使用 ECharts 图表组件渲染展示。

5 系统测试

5.1 测试策略

系统采用多层次测试策略：后端使用 JUnit 5 + Mockito 进行单元测试和集成测试，H2 内存数据库作为测试环境；前端使用 Vitest + Vue Test Utils 进行组件和 Store 测试；全量 API 通过 Postman 集合进行接口验证。

5.2 测试用例设计

系统采用黑盒测试法，针对五大核心模块设计了共计 32 个测试用例，覆盖各模块的主要功能路径、异常场景和边界条件。以下按模块逐一列出。

5.3 测试结果

5.3.1 单元测试

后端共完成 37 个单元测试，覆盖 Result 工具类（7 个）、JwtUtil 工具类（3 个）、UserServiceImpl（16 个）、AddressServiceImpl（8 个）、Controller 集成测试（2 个）以及上下文加载测试（1 个）；前端共完成 13 个单元测试，覆盖 auth 认证工具函数（3 个）、组件渲染（2 个）、Result 数据格式（3 个）以及 Pinia Store 状态管理（5 个）。全部 50 个测试均通过，成功率 100%。

表 3: 用户模块测试用例

编号	测试项目	测试标题	级别	预期输出
USER_REG_001	用户注册	正常注册新用户	高	注册成功, 返回用户信息与 Token
USER_REG_002	用户注册	重复用户名注册	高	返回 400, 提示用户名已存在
USER_REG_003	用户注册	验证码错误注册	高	返回 400, 提示验证码错误
USER_LOGIN_001	用户登录	用户名密码正确登录	高	登录成功, 返回双 Token
USER_LOGIN_002	用户登录	密码错误登录	高	返回 401, 提示密码错误
USER_LOGIN_003	用户登录	已禁用账户登录	高	返回 403, 提示账户已禁用
USER_TOKEN_001	Token 刷新	正常刷新 Token	中	返回新的 Access Token
USER_RESET_001	密码重置	正常重置密码	中	重置成功, 可使用新密码登录
USER_INFO_001	个人信息	获取个人信息	低	返回当前用户详细信息
USER_ADDR_001	地址管理	新增地址(首个自动默认)	中	地址添加成功, 自动设为默认
USER_ADDR_002	地址管理	删除非本人地址	中	返回 403, 无权操作

表 4: 商品模块测试用例

编号	测试项目	测试标题	级别	预期输出
PROD_PUB_001	商品发布	正常发布商品	高	发布成功, 状态为"待审核"
PROD_PUB_002	商品发布	未登录发布商品	高	返回 401, 提示未认证
PROD_LIST_001	商品列表	分页查询商品列表	中	返回分页商品数据
PROD_SEARCH_001	商品搜索	按分类 + 价格 + 排序搜索	中	返回筛选排序后的商品列表
PROD_SEARCH_002	商品搜索	关键词全文搜索	中	返回匹配关键词的商品列表
PROD_FAV_001	商品收藏	收藏商品	低	收藏成功, 可查询收藏列表
PROD_FAV_002	商品收藏	重复收藏同一商品	低	返回 400, 提示已收藏
PROD_EDIT_001	商品编辑	卖家编辑自己商品	中	编辑成功, 状态重置为待审核
PROD_DEL_001	商品删除	卖家删除自己商品	中	删除成功(软删除)

表 5: 购物车模块测试用例

编号	测试项目	测试标题	级别	预期输出
CART_ADD_001	添加购物车	正常添加商品到购物车	中	添加成功, 返回购物车项
CART_ADD_002	添加购物车	添加已存在的商品	中	数量增加, 不重复创建
CART_LIST_001	购物车列表	查询购物车列表	中	返回当前用户购物车列表
CART_UPDATE_001	更新数量	修改购物车商品数量	低	更新成功
CART_DEL_001	删除购物车项	删除购物车中商品	低	删除成功

表 6: 订单模块测试用例

编号	测试项目	测试标题	级别	预期输出
ORDER_CREATE_001	创建订单	正常创建订单	高	创建成功, 状态为”待付款”
ORDER_CREATE_002	创建订单	商品不存在或已下架	高	返回 404, 商品不可购买
ORDER_PAY_001	订单支付	正常完成支付	高	支付成功, 状态变为”待发货”
ORDER_PAY_002	订单支付	重复支付已支付订单	中	返回 400, 订单状态异常
ORDER_SHIP_001	卖家发货	录入物流信息发货	高	发货成功, 状态变为”待收货”
ORDER_RECV_001	买家收货	确认收货	高	收货成功, 状态变为”已完成”
ORDER_CANCEL_001	取消订单	待付款状态取消订单	中	取消成功, 状态变为”已取消”
ORDER_REVIEW_001	订单评价	完成订单后评价	中	评价成功, 状态变为”已评价”
ORDER_LOG_001	订单日志	查询订单操作日志	低	返回订单状态变更记录

表 7: 消息模块测试用例

编号	测试项目	测试标题	级别	预期输出
MSG_SEND_001	发送消息	发送文本消息	中	消息发送成功
MSG_SEND_002	发送消息	给自己发送消息	低	发送成功(或提示不允许)
MSG_LIST_001	会话列表	查询用户会话列表	中	返回所有会话及最后一条消息
MSG_HIST_001	聊天记录	分页查询聊天记录	中	返回分页消息记录
MSG_READ_001	已读标记	标记消息为已读	低	标记成功, 未读数减少
NOTIF_LIST_001	通知列表	查询系统通知列表	低	返回通知列表

表 8: 后台管理模块测试用例

编号	测试项目	测试标题	级别	预期输出
ADMIN_USER_001	用户管理	查询所有用户列表	中	返回分页用户列表
ADMIN_USER_002	用户管理	禁用/启用用户	中	用户状态更新成功
ADMIN_PROD_001	商品审核	审核通过商品	高	审核通过, 商品状态变更为”在售”
ADMIN_PROD_002	商品审核	审核驳回商品	高	驳回成功, 商品返回”审核不通过”
ADMIN_ORDER_001	订单管理	查询全部订单	中	返回分页订单列表
ADMIN_STAT_001	数据统计	查看统计数据	低	返回用户量、商品量、成交量等统计
ADMIN_ANN_001	公告管理	发布系统公告	低	公告发布成功, 用户可查看

表 9: 安全模块测试用例

编号	测试项目	测试标题	级别	预期输出
SEC_AUTH_001	权限控制	未认证访问需认证接口	高	返回 401 未认证
SEC_AUTH_002	权限控制	普通用户访问管理接口	高	返回 403 无权限
SEC_AUTH_003	权限控制	已认证用户访问公开接口	高	返回正常数据
SEC_TOKEN_001	Token 校验	使用过期 Token 访问接口	高	返回 401 Token 过期

表 10: 单元测试统计

测试范围	测试数	通过	通过率
后端单元测试	37	37	100%
前端单元测试	13	13	100%
总计	50	50	100%

5.3.2 接口测试

通过 Postman 集合对全部 60+ 个 API 端点进行验证, 涵盖首页 (3 个)、用户认证 (6 个)、用户信息 (3 个)、地址管理 (5 个)、商品 (11 个)、购物车 (4 个)、订单 (6 个)、消息 (5 个)、管理后台 (8 个)、上传 (1 个) 十大模块, 全部接口通过验证。

5.3.3 性能优化

在测试过程中发现并修复了以下关键问题:

1. **N+1 查询问题:** 商品列表每页加载 41 条 SQL 语句, 通过批量加载分类和卖家信息优化至 3 条 SQL, 性能提升约 13 倍。
2. **SQL 注入漏洞:** ‘ProductMapper.xml’中排序字段直接拼接参数, 改为白名单校验机制, 仅允许预定义字段参与排序。
3. **数据库索引缺失:** 新增 4 个复合索引, 分别优化消息会话查询 (from_user_id+to_user_id+created_at)、未读计数 (to_user_id+is_read)、收藏排序 (user_id+created_at) 和卖家商品筛选 (user_id+status)。
4. **逻辑错误:** ‘countProductByUser()’查错表, 修正为正确的 Mapper 方法。

6 结论与体会

6.1 结论

经过为期一周的集中开发与测试, ”转转” 校园二手物品交易平台已成功实现全部预设功能。系统基于 Spring Boot 3.2.0 + Vue 3.4 前后端分离架构, 实现了用户管理、商品管理、购物车与订单交易、站内消息通信、后台管理五大核心模块, 共计 60 余个 RESTful API 和 23 个前端页面 (21 条路由)。系统通过了 50 个单元测试 (后端 37 个、前端 13 个) 和全量 Postman 接口测试, 成功率为 100%, 并已通过 Docker

Compose 实现五容器一键部署。

项目的核心成果包括：实现了基于 JWT 双 Token 机制的安全认证体系，保障了接口访问安全；设计了完整的订单状态流转引擎，支持从下单到评价的全流程闭环；构建了基于 Redis 的缓存加速机制，提升了系统响应性能；同时修复了 N+1 查询、SQL 注入等关键质量和安全问题。

6.2 个人体会

通过本次综合实训，我在以下几个方面获得了显著提升：

6.2.1 系统分析与设计能力

在需求分析阶段，学习如何将模糊的业务需求转化为明确的系统功能和数据结构；在架构设计阶段，深入理解了前后端分离架构的优势，掌握了 RESTful API 的设计规范和数据库表设计的范式理论。

6.2.2 技术实践能力

通过实际开发，巩固了 Spring Boot 框架的核心特性（IoC、AOP、事务管理），熟练掌握了 MyBatis-Plus 的 Lambda 查询和分页插件使用，理解了 JWT 无状态认证的工作原理和实现方式。前端方面，深入体验了 Vue 3 组合式 API 的开发模式，掌握了 Pinia 状态管理和 Axios 拦截器等关键技术。

6.2.3 项目管理与团队协作

作为队长，负责全组任务分配、进度协调和最终质量把控。使用 Git 进行版本控制和分支管理，确保多人协作的代码一致性。每日召开晨会同步进度，及时解决联调中遇到的技术问题。

6.2.4 问题解决能力

在开发过程中遇到了一系列技术挑战——Docker 容器启动顺序控制、订单库存扣减的原子性保证、前后端字段命名不一致等。通过查阅官方文档和技术社区，逐一找到了合理的解决方案，积累了宝贵的实战经验。

6.2.5 质量意识

测试环节让我深刻认识到软件质量的重要性。通过多层次测试策略，发现了 N+1 查询性能瓶颈和 SQL 注入安全漏洞等容易被忽视的问题，意识到好的软件不仅要“能用”，更要“好用”、“安全”。

7 参考文献

1. 尤雨溪. Vue.js 3 设计与实现 [M]. 北京: 人民邮电出版社, 2023.
2. Walls C. Spring Boot 实战 [M]. 第 4 版. 北京: 人民邮电出版社, 2022.
3. Widenius M. MySQL 必知必会 [M]. 北京: 人民邮电出版社, 2020.
4. 黄健宏. Redis 设计与实现 [M]. 北京: 机械工业出版社, 2014.

-
5. Richardson L, Ruby S. RESTful Web APIs[M]. O'Reilly Media, 2013.
 6. 王松. Spring Boot 整合开发实战 [M]. 北京: 清华大学出版社, 2023.
 7. 张卫朋. Vue.js 3 从入门到精通 [M]. 北京: 人民邮电出版社, 2024.
 8. 孙宏明. MyBatis-Plus 从入门到精通 [M]. 北京: 清华大学出版社, 2023.
 9. 李刚. 轻量级 Java EE 企业应用实战 [M]. 第 6 版. 北京: 电子工业出版社, 2024.